

Sprint #0 Report

Instructions

Objectives

- Make decisions on the Solitaire software development project.
- Learn unit testing and GUI programming in the language of your choice.

Important Note: I had some issue with pushing it onto github, as I was accidentally using my old account “CaldeJE2290” and my current github “jecalvil”, so it shows two contributors, of which both are mine. This then resulted in some issues with the branching. It is seemingly fixed, just make sure the “main” branch is being used and not the “master”

Deliverables and Grading Policy

Read the “CS 449 Homework Overview” document **carefully** and make the key decisions for the software development. Use the following template to complete your report.

1. Key Decisions of the Solitaire Project (2 points)

Topic	Decision
Object-oriented programming language (if not using OOP, then justify the use of the language)	Python 3.14.3 (OOP)
GUI library (strongly encouraged)	Pygame-CE 2.5.8
IDE (Integrated Development Environment)	Visual Studio Code
xUnit framework (e.g., JUnit for Java)	Unittest (Python built in standard library)
Programming style guide (must read it carefully)	Google Python Style Guide
Project hosting site	Github.com
Other decisions if applicable	Planned Enhancements: User “theme” selection, background music, sound effects, optional move hints

Python was chosen as it is the language I have worked with most recently, its readable syntax, and its built in unittest. Pygame CE provides all that is needed for this board GUI (mouse input, buttons, etc). I am following the Google Python Style Guide as suggested below, using module imports and docstrings.

Sample programming style guides:

- Google Java Style Guide: <https://google.github.io/styleguide/javaguide.html>
- Google C++ Style Guide: <https://google.github.io/styleguide/cppguide.html>
- Google Python Style Guide: <https://google.github.io/styleguide/pyguide.html>

2. Unit testing (4 points)

Find a tutorial on the unit test framework you have chosen and write at least two xUnit tests of a program you have written or found elsewhere. Attach here (1) the screenshot of your program execution and (2) the source code of your program.

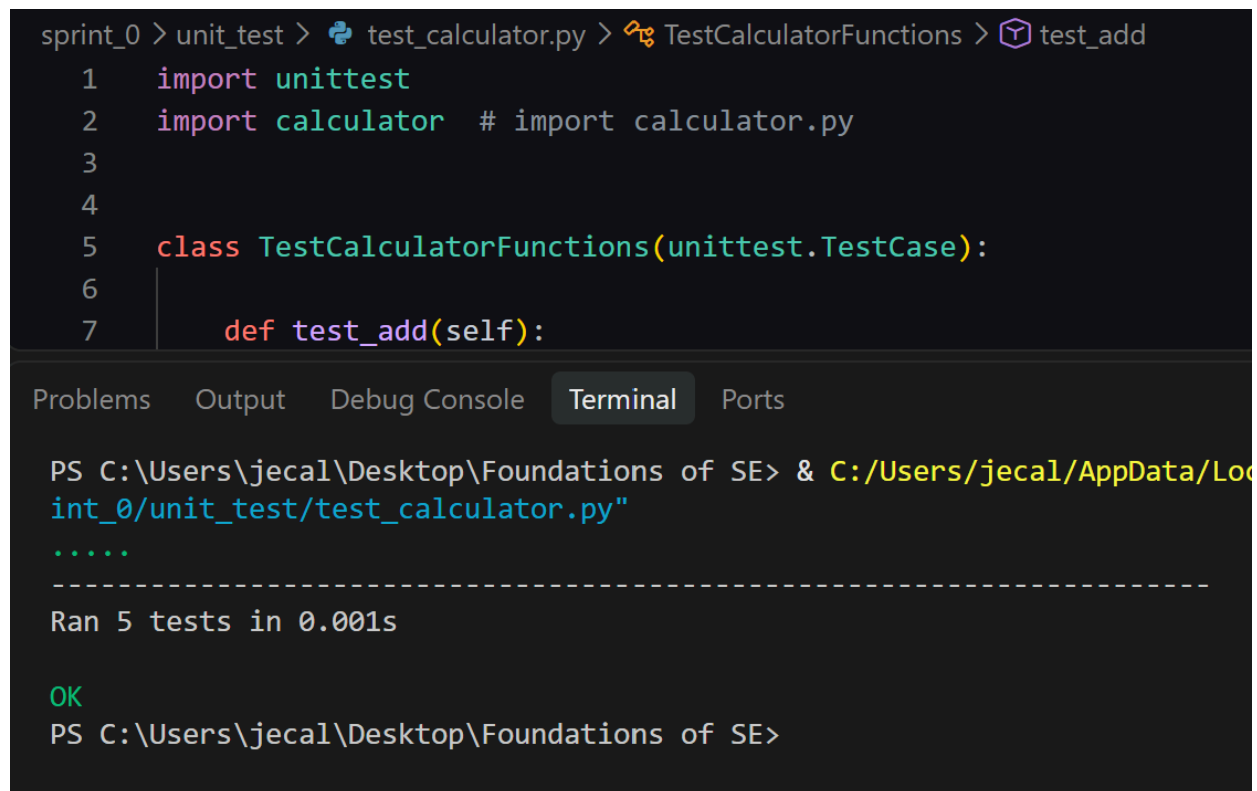
For example, Junit5 user guide: <https://docs.junit.org/5.10.3/user-guide/index.html>

Tutorial used for python unittesting: <https://www.youtube.com/watch?v=6tNS--WetLI>

Calculator app source code found on the internet for writing unit tests:
<https://www.geeksforgeeks.org/python/make-simple-calculator-using-python/>

My code on my github for the unit tests:
https://github.com/jecalvil/Sprint-0/blob/main/sprint_0/unit_test/test_calculator.py

Screenshot of program execution:



The screenshot shows an IDE with a Python file named `test_calculator.py` open. The file contains the following code:

```
sprint_0 > unit_test > test_calculator.py > TestCalculatorFunctions > test_add
1  import unittest
2  import calculator # import calculator.py
3
4
5  class TestCalculatorFunctions(unittest.TestCase):
6
7      def test_add(self):
```

The terminal output shows the execution of the test:

```
PS C:\Users\jecal\Desktop\Foundations of SE> & C:/Users/jecal/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe C:/Users/jecal/Desktop/Foundations of SE/unit_0/unit_test/test_calculator.py
.....
-----
Ran 5 tests in 0.001s

OK
PS C:\Users\jecal\Desktop\Foundations of SE>
```

Code For The Unit Test:

Note: I clarified with the professor that each method within the class counts as 1 unit test, so 5 in total, rather than the whole class for the calculator function counting as 1

```
import unittest
import calculator # import calculator.py

class TestCalculatorFunctions(unittest.TestCase):
    """Unit tests for the calculator functions."""

    def test_add(self):
```

```

        self.assertEqual(calculator.add(10, 5), 15)
        self.assertEqual(calculator.add(-1, 1), 0)
        self.assertEqual(calculator.add(-2, -2), -4)

    def test_sub(self):
        self.assertEqual(calculator.sub(5, 3), 2)
        self.assertEqual(calculator.sub(-1, 1), -2)
        self.assertEqual(calculator.sub(-1, -1), 0)

    def test_div(self):
        self.assertEqual(calculator.div(10, 2), 5)
        self.assertEqual(calculator.div(10, -2), -5)

    def test_mul(self):
        self.assertEqual(calculator.mul(5, 5), 25)
        self.assertEqual(calculator.mul(5, -5), -25)
        self.assertEqual(calculator.mul(-5, -5), 25)
        self.assertEqual(calculator.mul(5, 0), 0)

    def test_divide_by_zero(self):
        with self.assertRaises(ZeroDivisionError):
            calculator.div(10, 0)

if __name__ == "__main__":
    unittest.main()

```

Code for Calculator Program:

```

# Code is sourced from geeksforgeeks.org
# Original Code Linked:
https://www.geeksforgeeks.org/python/make-simple-calculator-using-python/
# Titled: "Command-Line Calculator" on website
# Tutorial watched: https://www.youtube.com/watch?v=6tNS--WetLI
# Code is slightly modified for unittest (line 25 with if __name__... and indentations below it)

def add(n1, n2):
    return n1 + n2

def sub(n1, n2):
    return n1 - n2

```

```

def mul(n1, n2):
    return n1 * n2

def div(n1, n2):
    return n1 / n2

# Line 25 added in order to do unittesting
if __name__ == "__main__":
    print(
        "Please select operation -\n"
        "1. Add\n"
        "2. Subtract\n"
        "3. Multiply\n"
        "4. Divide\n"
    )

    sel = int(input("Select operation (1-4): "))

    n1 = int(input("Enter first number: "))
    n2 = int(input("Enter second number: "))

    if sel == 1:
        print(n1, "+", n2, "=", add(n1, n2))
    elif sel == 2:
        print(n1, "-", n2, "=", sub(n1, n2))
    elif sel == 3:
        print(n1, "*", n2, "=", mul(n1, n2))
    elif sel == 4:
        print(n1, "/", n2, "=", div(n1, n2))
    else:
        print("Invalid input")

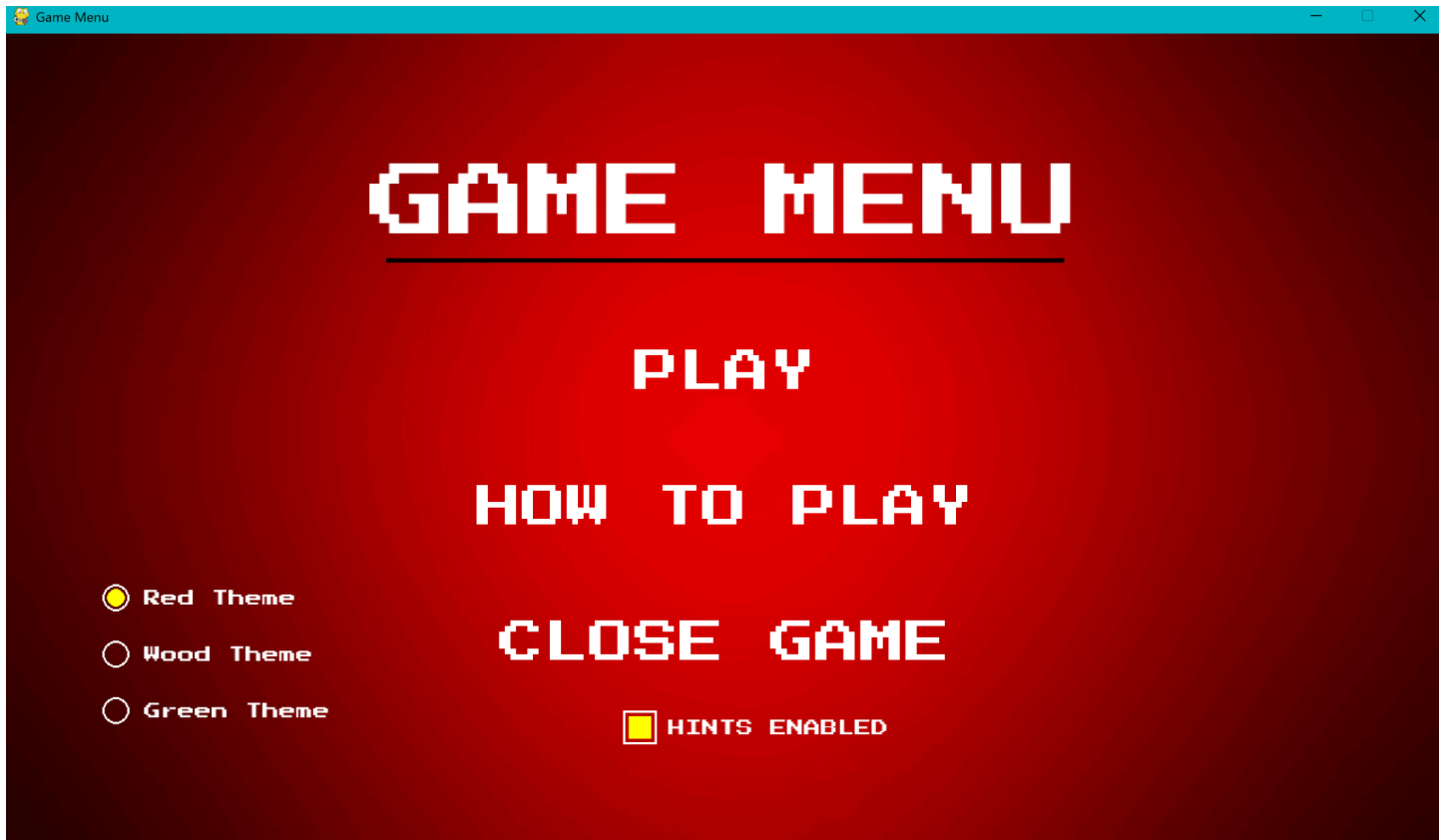
```

3. GUI programming (4 points)

Write a GUI program in the language you have chosen for your Solitaire project. The GUI of your program must include text, lines, a check box, and radio buttons. In this assignment, any GUI program of your own work is acceptable.

Attach here (1) the screenshot of your program execution and (2) the source code of your program.

1) Screenshot of program execution:



- 2) Source code link on github:
https://github.com/jecalvil/Sprint-0/blob/main/sprint_0/gui_programming/gui.py

gui.py:

```
"""Main menu GUI for the Solitaire game."""

import os
import sys

import pygame

import components

# Requirements:
# Line, Check box, Text

pygame.init()

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

# -- Paths --
# Theme Paths
THEME1_PATH = os.path.join(SCRIPT_DIR, "images", "red_background.jpg")
```

```

THEME2_PATH = os.path.join(SCRIPT_DIR, "images", "wood_background.jpg")
THEME3_PATH = os.path.join(SCRIPT_DIR, "images", "green_background.jpg")
# Font Paths
FONT_PATH = os.path.join(SCRIPT_DIR, "PressStart2P-Regular.ttf")

# Create screen
SCREEN = pygame.display.set_mode((1280, 720))
pygame.display.set_caption("Game Menu")

# Load Themes
THEME1 = pygame.image.load(THEME1_PATH)
THEME2 = pygame.image.load(THEME2_PATH)
THEME3 = pygame.image.load(THEME3_PATH)

# Make background image scale
THEME1 = pygame.transform.scale(THEME1, (1280, 720))
THEME2 = pygame.transform.scale(THEME2, (1280, 720))
THEME3 = pygame.transform.scale(THEME3, (1280, 720))

# Fonts
font = pygame.font.Font(FONT_PATH, 40)
title_font = pygame.font.Font(FONT_PATH, 70)
checkbox_font = pygame.font.Font(FONT_PATH, 15)

def main_menu():
    """Runs the main menu loop until the player closes the game."""
    # Hints_Enabled? - Checkbox requirement
    hints_toggle = False
    # Initial Theme
    selected_theme = THEME1

    # Menu Buttons
    play_button = components.Button("PLAY", font, "White", "Yellow", (640, 300))
    rules_button = components.Button("HOW TO PLAY", font, "White", "Yellow", (640,
420))
    close_button = components.Button("CLOSE GAME", font, "White", "Yellow", (640,
540))

    # -- Radio Buttons --
    # x-coordinate
    radio1_x = 100
    radio2_x = radio1_x
    radio3_x = radio1_x

```

```

# y coordinate
radio1_y = 500
radio2_y = radio1_y + 50
radio3_y = radio2_y + 50

# Create radio buttons
radio_buttons = [
    components.RadioButton(
        radio1_x, radio1_y, 12, "Red Theme", THEME1, checkbox_font
    ),
    components.RadioButton(
        radio2_x, radio2_y, 12, "Wood Theme", THEME2, checkbox_font
    ),
    components.RadioButton(
        radio3_x, radio3_y, 12, "Green Theme", THEME3, checkbox_font
    ),
]

# Main menu event and rendering loop
while True:
    # Draw background image
    SCREEN.blit(selected_theme, (0, 0))

    # Current Mouse Position
    MENU_MOUSE_POS = pygame.mouse.get_pos()

    # -- Title Text --
    title_surface = title_font.render("GAME MENU", True, "White")
    title_rect = title_surface.get_rect(center=(640, 150))
    SCREEN.blit(title_surface, title_rect)

    # Divider Line (Line Requirement)
    pygame.draw.line(SCREEN, "Black", (340, 200), (940, 200), 4)

    # -- Draw Menu Buttons --
    play_button.draw(SCREEN, MENU_MOUSE_POS)
    rules_button.draw(SCREEN, MENU_MOUSE_POS)
    close_button.draw(SCREEN, MENU_MOUSE_POS)

    # -- Draw Radio Buttons --
    for radio_button in radio_buttons:
        radio_button.draw(SCREEN, selected_theme)

    # -- Create Checkbox --

```

```

# Note: In the future, likely make a class if multiple are needed
hints_rect = pygame.Rect(550, 600, 30, 30)
pygame.draw.rect(SCREEN, "White", hints_rect, 2)
# toggle checkbox
checkbox_text = "HINTS DISABLED"
if hints_toggle:
    pygame.draw.rect(SCREEN, "Yellow", pygame.Rect(555, 605, 20, 20))
    checkbox_text = "HINTS ENABLED"

# checkbox label
label_surface = checkbox_font.render(checkbox_text, True, "White")
SCREEN.blit(label_surface, (590, 608))

# -- Event Loop --
for event in pygame.event.get():
    # Default X window button click
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()

    # Mouse Clicks
    if event.type == pygame.MOUSEBUTTONDOWN:

        # Menu Buttons
        if play_button.check_click(MENU_MOUSE_POS):
            print("Play button clicked") # Add Functionality Later

        if rules_button.check_click(MENU_MOUSE_POS):
            print("Rules button clicked") # Add Functionality Later

        if close_button.check_click(MENU_MOUSE_POS):
            print("Game closed")
            pygame.quit()
            sys.exit()

        # Menu Checkbox
        if hints_rect.collidepoint(event.pos):
            hints_toggle = not hints_toggle

        # Radio Button Clicks
        if radio_buttons[0].check_click(event.pos):
            selected_theme = radio_buttons[0].value
        elif radio_buttons[1].check_click(event.pos):
            selected_theme = radio_buttons[1].value

```

```

        elif radio_buttons[2].check_click(event.pos):
            selected_theme = radio_buttons[2].value

    # Refresh screen display
    pygame.display.update()

# Run menu
if __name__ == "__main__":
    main_menu()

```

components.py:

```

"""Reusable GUI components for the game menu."""

import pygame

class Button:
    """A clickable text button that changes color upon hovering over."""

    def __init__(self, text, font, text_color, hover_color, center_pos):
        self.text = text
        self.font = font
        self.text_color = text_color
        self.hover_color = hover_color
        self.center_pos = center_pos
        self.surface = self.font.render(self.text, True, self.text_color)
        self.rect = self.surface.get_rect(center=self.center_pos)

    def draw(self, screen, mouse_pos):
        """Draws the button, using the hover color if the mouse is over it."""
        if self.rect.collidepoint(mouse_pos):
            hover_surface = self.font.render(self.text, True, self.hover_color)
            screen.blit(hover_surface, self.rect)
        else:
            screen.blit(self.surface, self.rect)

    def check_click(self, mouse_pos):
        """Returns True if mouse_pos is inside the button."""
        return self.rect.collidepoint(mouse_pos)

```

```
class RadioButton:
    """A circular radio button with a text label."""

    def __init__(self, x, y, radius, label, value, font):
        self.center = (x, y)
        self.radius = radius
        self.label = label
        self.value = value
        self.font = font
        self.label_surface = self.font.render(self.label, True, "White")
        self.label_rect = self.label_surface.get_rect(midleft=(x + 25, y))

    def draw(self, screen, selected_value):
        """Draws the button, filled in if its value is the selected one."""
        pygame.draw.circle(screen, "White", self.center, self.radius, 2)
        if selected_value == self.value:
            pygame.draw.circle(screen, "Yellow", self.center, self.radius - 4)
        screen.blit(self.label_surface, self.label_rect)

    def check_click(self, mouse_pos):
        """Returns True if mouse_pos is inside the radio button's circle."""
        return pygame.Vector2(mouse_pos).distance_to(self.center) <= self.radius
```